

Persistencia Backend : Base de Datos y Modelos

1. Introducción

Para construir aplicaciones backend robustas con FastAPI es fundamental diseñar correctamente la capa de persistencia. Esto implica:

- Configurar adecuadamente la conexión a la base de datos.
- Definir modelos bien estructurados.
- Separar responsabilidades entre almacenamiento y exposición de datos.
- Controlar el flujo de transacciones.

En esta unidad se utiliza **SQLModel**, una librería que combina:

- Validación de datos con Pydantic.
- ORM (Object Relational Mapping) basado en SQLAlchemy.
- Integración directa con FastAPI.

SQLModel permite definir modelos que sirven tanto como representación de tablas de base de datos como esquemas de validación, reduciendo duplicación de código y manteniendo claridad arquitectónica.

La base de datos utilizada es **PostgreSQL**, un sistema de gestión relacional orientado a producción.

2. Flujo de la Base de Datos

Estructura de database.py

La capa de persistencia centraliza la configuración de conexión y la gestión de sesiones.

El flujo general es:

Cliente → Endpoint → Modelo de entrada → Lógica → Modelo DB → Session →
Engine → PostgreSQL → Commit → Modelo público → Respuesta

Cada componente cumple una responsabilidad específica.

2.1 El Engine (Motor de Conexión)

El **engine** es el objeto que administra las conexiones a la base de datos.
Debe existir una única instancia global para toda la aplicación.

Configuración con PostgreSQL:

```
from sqlmodel import create_engine

import os

DATABASE_URL = os.getenv (

    "DATABASE_URL",

    "postgresql+psycopg://user:password@localhost:5432/database_name"

)

engine = create_engine(DATABASE_URL, echo=True)
```

Explicación:

- postgresql+psycopg indica el dialecto y el driver.
- echo=True permite visualizar las consultas SQL generadas (útil en desarrollo).
- Se recomienda usar variables de entorno en lugar de credenciales hardcodeadas.

2.2 Creación de Tablas

Para que los modelos definidos en código se reflejen en la base de datos:

```
from sqlmodel import SQLModel

def create_db_and_tables():

    SQLModel.metadata.create_all(engine)
```

Esta función normalmente se ejecuta en el evento startup de FastAPI.

Conceptualmente:

- SQLAlchemy inspecciona los modelos declarados.
- Genera las sentencias SQL necesarias.
- Crea las tablas si no existen.

2.3 La Sesión (Session)

La **Session** representa una transacción activa con la base de datos.

Responsabilidades:

- Mantener objetos en memoria.
- Detectar cambios.
- Ejecutar transacciones.
- Confirmar (commit) o revertir (rollback).

Concepto clave:

Mientras no se ejecute `commit()`, los cambios no son permanentes.

2.4 Inyección de Dependencias

FastAPI permite crear una sesión por cada petición HTTP:

```
from typing import Annotated

from fastapi import Depends

from sqlalchemy import Session

def get_session():

    with Session(engine) as session:

        yield session

SessionDep = Annotated[Session, Depends(get_session)]
```

Ventajas:

- Cada request tiene su propia transacción.
- Se evita compartir estado entre usuarios.
- Se garantiza cierre correcto de la sesión.

3. Modelos Segregados (Separación de Responsabilidades)

Permitir que el frontend interactúe directamente con el modelo completo de base de datos es un error arquitectónico y de seguridad.

SQLModel permite herencia para dividir modelos en:

- Modelo base compartido.
- Modelo de tabla (persistencia).
- Modelos de entrada y salida (API).

Esto cumple principios como:

- Separación de capas.
- Responsabilidad única.
- Desacople entre API y almacenamiento.

3.1 Clase Base

Contiene los campos comunes:

```
from sqlmodel import Field, SQLModel

class HeroBase(SQLModel):

    name: str = Field(index=True)

    age: int | None = Field(default=None, index=True)
```

No representa tabla, solo estructura compartida.

3.2 Modelo de Tabla

Representa la entidad persistida:

```
from typing import Optional

from sqlalchemy import SQLModel, Field

class Hero(SQLModel, table=True):

    id: Optional[int] = Field(default=None, primary_key=True)

    name: str

    secret_name: str

    age: Optional[int] = None
```

Características:

- `table=True` indica que se mapeará a una tabla.
- `id` es clave primaria autogenerada.
- Incluye campos internos que no deben exponerse públicamente.

3.3 Modelos de Datos (API)

En una arquitectura correctamente diseñada, los modelos que representan las tablas de la base de datos no deben utilizarse directamente como esquemas de entrada y salida en la API.

Esto se debe a tres razones fundamentales:

1. Seguridad: no todos los campos deben exponerse públicamente.
2. Control: el cliente no debe poder modificar campos críticos como identificadores.

3. Evolución: la API puede cambiar sin necesidad de alterar la estructura interna de la base de datos.

Por este motivo se definen modelos específicos para cada tipo de interacción.

Modelo de creación (HeroCreate)

Este modelo representa los datos que el cliente debe enviar al crear un recurso. Incluye los campos necesarios para registrar la entidad, pero no incluye el identificador, ya que este es generado automáticamente por la base de datos.

Modelo público (HeroPublic)

Este modelo define qué información se devuelve al cliente. Incluye el identificador generado, pero excluye campos sensibles como `secret_name`. De esta forma, aunque el modelo de base de datos contenga información privada, la API actúa como filtro.

Modelo de actualización (HeroUpdate)

Este modelo se utiliza para operaciones PATCH. Todos sus campos se declaran como opcionales para permitir actualizaciones parciales. El sistema solo modificará los campos que el cliente haya enviado explícitamente.

Conceptualmente, el flujo es el siguiente:

El cliente envía datos → se validan con un modelo de entrada → se transforman en modelo de base de datos → se guardan → se devuelve un modelo público filtrado.

Este diseño evita que la estructura interna de la base de datos quede expuesta directamente al exterior.

4. Flujo DB en las Rutas (APIRouter)

FastAPI permite organizar endpoints usando APIRouter.

```
from fastapi import APIRouter

router = APIRouter(prefix="/heroes", tags=["heroes"])
```

4.1 Creación de Registros

```
@router.post("/", response_model=HeroPublic)

def create_hero(hero: HeroCreate, session: SessionDep):
```

```
db_hero = Hero.model_validate(hero)

session.add(db_hero)

session.commit()

session.refresh(db_hero)

return db_hero
```

Flujo técnico:

1. Se recibe HeroCreate.
2. Se transforma en modelo de tabla.
3. add() registra el objeto.
4. commit() ejecuta la transacción.
5. refresh() sincroniza el objeto (ej: obtiene el id generado).
6. response_model filtra datos sensibles.

4.2 Actualización Parcial (PATCH)

Diferencia conceptual:

- PUT → reemplaza completamente el recurso.
- PATCH → modifica solo campos enviados.

```
from fastapi import HTTPException

@router.patch("/{hero_id}", response_model=HeroPublic)

def update_hero(hero_id: int, hero: HeroUpdate, session: SessionDep):

    hero_db = session.get(Hero, hero_id)

    if not hero_db:

        raise HTTPException(status_code=404, detail="Hero not found")
```

```
hero_data = hero.model_dump(exclude_unset=True)

hero_db.sqlmodel_update(hero_data)

session.add(hero_db)

session.commit()

session.refresh(hero_db)

return hero_db
```

exclude_unset=True evita sobrescribir campos no enviados.

5. Transacciones y Manejo de Errores

Una sesión gestiona una transacción.

Propiedades fundamentales:

- Atomicidad: o todo se ejecuta o nada.
- Consistencia: mantiene reglas de integridad.
- Aislamiento: transacciones no interfieren.
- Durabilidad: tras commit, los cambios son permanentes.

Ejemplo con rollback:

```
try:

    session.commit()

except Exception:

    session.rollback()

raise
```

Esto evita dejar la base en estado inconsistente.

6. Relaciones entre Modelos

La persistencia real incluye relaciones.

Ejemplo uno a muchos:

```
from sqlmodel import Relationship

class Team(SQLModel, table=True):

    id: int | None = Field(default=None, primary_key=True)

    name: str

    heroes: list["Hero"] = Relationship(back_populates="team")
```

```
class Hero(HeroBase, table=True):

    id: int | None = Field(default=None, primary_key=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")

    team: Team | None = Relationship(back_populates="heroes")
```

Conceptos clave:

- foreign_key define integridad referencial.
- Relationship() permite navegación entre objetos.
- PostgreSQL asegura coherencia entre tablas.